

作业 HW1* 实验报告

姓名：李盛鹏 学号：2351136 日期：2024 年 9 月 29 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景

- 线性表：可以使用数组或链表来存储多项式的系数，便于访问和修改。
- 数组或列表：用于存储项的值，便于逐项累加。
- 栈：适合实现“后进先出”的牌堆操作。
- 队列：适合实现“先进先出”的操作。（当然栈和队列虽然涉及，但是不能使用）
- 时间复杂度：加法和乘法操作通常为 $O(n)$ ，其中 n 为多项式的最高次幂。

2. 实验内容

2.1 最大子段和（题目名字）

1. 轮转数组

2.1.1 问题描述

- # 简短地描述题目（可复制题目要求）
- 给定一个整数顺序表 `nums`，需要将其中的元素向右轮转 k 个位置，其中 k 是一个非负整数。

2.1.2 基本要求

简短地描述题目要求

输入格式：

- 第一行包含两个整数 n 和 k ，分别表示 `nums` 中的元素个数和向右轮转的位置数。
- 第二行包含 n 个整数，表示顺序表 `nums` 中的元素。

输出格式：

输出轮转后的顺序表中的元素，按顺序排列。

约束条件：

n 为顺序表的长度， k 为非负数。若 k 大于 n ，应考虑轮转的有效位置（例如， $k \% n$ ）。

2.1.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

例如

```
// Definition for a binary tree node.
struct TreeNode {
    int data;          // 节点元素
    TreeNode *lchild;  // 左孩子节点指针
    TreeNode *rchild;  // 右孩子指针
    TreeNode(int x = 0, TreeNode *left = nullptr, TreeNode *right = nullptr);
    ~TreeNode();
};
```

```
// 定义数组轮转函数的相关数据结构和功能
struct RotateArray {
    int data[MAX_SIZE]; // 存储顺序表中的元素
    int num;             // 数组元素个数
    int times;           // 向右轮转的次数

    // 构造函数：初始化数组和参数
    RotateArray(int n = 0, int k = 0) : num(n), times(k) {
        for (int i = 0; i < num; i++) {
            data[i] = 0; // 初始化数组元素
        }
    }
}
```

2.1.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

例如

```
/**
 * @brief      二叉树指令构造
 * @param   order   构造指令列表
 * @param   index   当前节点序号
 * @return    子树根节点指针
 */
TreeNode *buildTreeDFS(const std::vector<int> &order, int &index) {
    if (子树为空) {
        返回空指针 // 返回空指针给父节点
    }
    // 构造以当前节点为根节点的子树并返回
    return new TreeNode(order[index ++], buildTreeDFS(order, index),
        buildTreeDFS(order, index));
}
```

```
}
```

```
函数 rotate(数组 b, 整数 num, 整数 times)
    如果 num == 0 或 times == 0 则
        返回
    结束如果

    声明 数组 dst, 大小为 num
    整数 distance = times 取模 num

    对于 i 从 0 到 num - 1 循环
        dst[(distance + i) 取模 num] = b[i]
    结束循环

    对于 i 从 0 到 num - 1 循环
        b[i] = dst[i]
    结束循环
结束函数

函数 main()
    整数 num, times
    声明 数组 a, 大小为 100000

    读取 num 和 times
    对于 i 从 0 到 num - 1 循环
        读取 a[i]
    结束循环

    调用 rotate(a, num, times)

    对于 i 从 0 到 num - 1 循环
        打印 a[i] 和空格
    结束循环
结束函数
```

```
函数 main()
    整数 num, times
    声明 数组 a, 大小为 100000

    读取 num 和 times
    对于 i 从 0 到 num - 1 循环
        读取 a[i]
    结束循环

    调用 rotate(a, num, times)

    对于 i 从 0 到 num - 1 循环
        打印 a[i] 和空格
    结束循环
结束函数
```

2.1.5 调试分析（遇到的问题解决方法）

简要描述调试过程

可以使用图片辅助说明

首先，我在理解数组旋转的具体逻辑上有些迷惑，特别是计算新的索引时，取模运算有时让我搞不清楚。其次，边界条件也是个挑战。比如当输入的数组大小为零或旋转次数为零时，我不太确定该如何在伪代码中合理处理这些情况，以避免错误。另外，我在写伪代码时，想要使逻辑清晰明了，但有时难以找到合适的表达方式，特别是在描述循环和输入输出部分的时候。

2.1.6 总结和体会

可以说说做此题的收获、分析这道题目的难点和易错点（数据边界、对算法效率的要求等）

收获：理解旋转逻辑：通过反转的方式实现数组旋转，掌握了一个高效且优雅的算法。这个技巧可以应用于其他类似的问题。

边界条件处理：增强了对边界条件的敏感性，了解如何在算法中考虑特殊情况（如空数组、旋转次数为 0 或负数等）。

难点：旋转次数的处理：理解旋转次数如何影响数组，尤其是当旋转次数大于数组长度时，进行模运算可能会让人困惑。

数组反转逻辑：对反转操作的实现需要清晰的理解，因为错误的索引或逻辑可能导致数组未按预期旋转。

2.2 题目二

一元多项式的加法和乘法运算

2.2.1 题目描述

本项目旨在实现一元多项式的加法和乘法运算。多项式使用有序线性表表示，每个元素包含两个数据项：系数和指数。输入的多项式根据指数递增有序排列。

2.2.2 题目要求

本项目要求实现一元多项式的加法和乘法运算，使用有序线性表表示多项式。输入包括两个多项式的项数及其系数和指数，后续输入指定运算类型（加法、乘法或同时进行两者）。输出要求按指数从小到大排列，且若结果为 0 0 则不输出任何内容。数据规模从 15 到 2050 项不等，确保程序在各类情况下有效运行，特别关注边界条件和输出格式。

2.2.3 数据结构设计

```
定义节点结构 Node
整数 value // 系数
整数 zhishu // 指数
Node* next // 指向下一个节点
```

Node 结构表示多项式中的一项，包括系数 (value)、指数 (zhishu) 和指向下一个节点的指针 (next)，以便形成链表结构。

2.2.4 功能实现

```

定义多项式类 duoxiangshi
属性
    Node* head // 多项式头指针

方法
    构造函数 duoxiangshi()
        初始化 head 为 nullptr

    析构函数 ~duoxiangshi()
        遍历节点, 释放内存

    方法 insert(value, zhishu)
        如果 value 为 0, 则返回
        创建新节点 newnode
        如果 head 为空
            设置 head 为 newnode
            返回
        遍历链表, 找到插入位置
        如果 newnode 的 zhishu >= 当前节点的 zhishu
            如果相等, 合并同类项
                如果合并后为 0, 则删除当前节点
            否则, 插入到当前节点前
        否则, 插入到合适的位置

```

```

方法 add(other, result)
    遍历两个多项式, 合并项并插入 result

方法 multiply(other, result)
    双重遍历两个多项式, 计算乘积并插入 result

方法 polynomial_print()
    遍历多项式并打印每个节点的 value 和 zhishu

```

duoxiangshi 类表示多项式, 包含一个指向头节点的指针 head。它提供构造函数和析构函数以初始化和释放内存, 以及插入项、加法和乘法运算的方法。通过遍历链表, 类支持合并同类项和打印多项式的功能。

2.2.5 心得体会

这道题的收获主要在于理解链表结构如何有效地表示和操作多项式, 掌握了动态内存管理和合并同类项的逻辑。同时, 编写和调试多项式的加法和乘法方法提升了代码组织能力。

遇到的困难包括: 1) 理解如何在链表中维护项的顺序和合并同类项; 2) 确保内存的正确分配和释放, 避免内存泄漏; 3) 实现复杂操作时, 保持代码的可读性和逻辑清晰性。

2.3 题目三

扑克牌

2.2.1 题目描述

这道题要求实现一个扑克牌堆的操作，包括添加、抽取、反转和弹出牌的功能。我们需要处理不同的命令，以维护牌堆的状态并输出相关结果。

2.2.2 题目要求

这道题要求实现一个扑克牌堆的操作系统，支持以下命令：添加（如 Append Spade A 将黑桃 A 添加到底部）、抽取（如 Extract Heart 抽取所有红心牌并按编号排序放到顶部）、反转（如 Revert 将整个牌堆逆序）和弹出（如 Pop 弹出顶部牌，如果为空则输出 NULL）。在处理完最多 200 个命令后，需输出每次弹出操作的结果，以及最终牌堆中剩余牌的顺序，若牌堆为空则输出 NULL。

2.2.3 数据结构设计

```
定义结构体 DuLNode
    属性:
        data // 存放扑克牌信息
        prior, next // 前驱和后继指针

定义类型 DuLinkList 为 DuLNode*
```

定义结构体 DuLNode，包含属性 data（存放扑克牌信息）以及 prior 和 next（分别指向前驱和后继节点），同时定义类型 DuLinkList 为指向 DuLNode 的指针。

2.2.4 功能实现

```
定义常量
    LOVERFLOW = -1
    ERROR = -2
    OK = 0

定义结构体 Poker
    属性:
        type[128] // 扑克牌花色
        num[4] // 扑克牌点数
    方法:
        getnum() // 返回牌的数值
            如果 num 为 'A' 返回 1
            如果 num 为 'J' 返回 11
            如果 num 为 'Q' 返回 12
            如果 num 为 'K' 返回 13
            如果 num 的第二位为 '0' 返回 10
            否则返回 num 的第一位的数字

    operator< // 按照点数降序比较
```

```
插入元素 InsertElem(DuLinkList& L, Elemtype e, bool from_head)
    分配内存给 newnode
    将 e 赋值给 newnode->data
    如果 from_head 为真
        更新指针，插入到链表头部
    否则
        更新指针，插入到链表尾部
    返回 OK

弹出元素 PopItem(DuLinkList& L, Elemtype& e, bool from_head)
    如果链表为空 (L->next == L)
        设置 e.type 为 "NULL"
        返回 OK
    如果 from head 为真
        获取链表头部元素
        更新链表指针
    否则
        获取链表尾部元素
        更新链表指针
    返回 OK
```



```

打印链表 PrintList(DuLinkedList L, bool from_head)
    设置指针 p 为头或尾
    遍历链表 (p != L)
        输出 p->data.type 和 p->data.num
        更新指针 p
    返回 OK

抽取元素 ExtractItem(DuLinkedList& L, char* type, bool from_head)
    设置指针 p 为头或尾
    初始化 vectorp = 0
    遍历链表 (p != L)
        如果 p->data.type 与 type 匹配
            从链表中移除节点
            将 p->data 保存到 vec[vectorp]
            vectorp++
    对 vec 按点数排序
    遍历 vec
        插入每张牌回链表
    返回 OK

```

该程序首先通过 InitList 初始化一个双向链表，并设置头尾指针指向自身。用户可以通过 InsertElem 函数在链表头部或尾部插入新的扑克牌元素。PopItem 函数用于从链表中弹出头部或尾部的元素，并在链表为空时处理特殊情况。PrintList 函数遍历链表，输出每张扑克牌的信息，方便用户查看当前状态。最后，ExtractItem 函数根据指定类型提取牌，并将其移除，同时对提取的牌按点数排序后重新插入链表。

2.2.5 心得体会

收获：在实现扑克牌堆操作系统的过程中，我获得了深刻的见解，特别是在数据结构的应用方面。通过设计和实现双向链表，我不仅加深了对链表基本操作的理解，还掌握了如何在动态内存管理中避免内存泄漏。这一过程促使我反思了算法的设计与实现，尤其是在抽取和排序功能的具体实现上，提升了我的算法思维和逻辑推理能力。此外，调试过程中的问题排查让我认识到细致测试的重要性，为后续的开发奠定了坚实的基础。

困难：然而，这一项目也带来了诸多挑战，特别是在指针操作的复杂性和特殊情况的处理上。链表的插入与删除操作需要小心翼翼地更新多个指针，任何疏忽都可能导致链表结构的破坏。同时，在面对链表为空或只有一个节点的边界情况时，确保逻辑的正确性常常令我倍感压力。此外，如何高效地实现元素的排序并将其正确地插入回链表，也是一项需要深入思考的任务。这些困难不仅锻炼了我的编程能力，更加深了我对数据结构和算法的理解。

2.4 题目四

求级数

2.2.1 题目描述

计算一个级数的和，其中级数的形式为 $S = A + 2A^2 + 3A^3 + \dots + N A^N = A + 2A^2 + 3A^3 + \dots + N A^N$ 。每一行输入包括整数 N 和 A ，需要根据这些值计算并输出对应的级数结果。

2.2.2 题目要求

1. 输入若干行数据，每行包含两个整数 NN 和 AA。
2. 输出对应的级数计算结果，格式为 S=结果 S=结果。
3. 保证计算结果为整数，并处理输入范围 $1 \leq N \leq 150$ 和 $0 \leq A \leq 15$ 。

2.2.3 数据结构设计

```
FUNCTION main()  
  INITIALIZE cal[152][200] TO 0  
  READ N, X
```

cal 是一个二维数组，初始化为全零，用于存储计算过程中生成的多项式系数和进位信息。

2.2.4 功能实现

```
FUNCTION make_list(my_list, X, N)  
  // 将 x 放入矩阵中  
  my_list[1][COLUMN - 1] = X % 10  
  my_list[1][COLUMN - 2] = X / 10  
  
  FOR i FROM 2 TO N DO  
    FOR j FROM 1 TO COLUMN - 1 DO  
      my_list[i][COLUMN - j] = my_list[i - 1][COLUMN - j] * X  
  
    // 处理进位  
    FOR j FROM COLUMN - 1 DOWN TO 1 DO  
      IF my_list[i][j] >= 10 THEN  
        my_list[i][j - 1] += my_list[i][j] / 10  
        my_list[i][j] = my_list[i][j] MOD 10
```

make_list 函数将输入值 XX 的个位和十位存储在 my_list 的第一行中，并通过逐行计算从第二行到第 NN 行的元素，使每个元素为上一行对应元素乘以 XX 的结果。在每次计算后，函数检查并处理进位，确保每个元素保持在单个数字范围内，从而有效地管理多项式的计算过程。

2.2.5 心得体会

收获：在实现 make_list 函数的过程中，我体会到了数据结构设计的重要性，尤其是在处理大数运算时，如何合理组织数据能够极大地简化计算逻辑。此外，逐步处理进位的过程让我深刻理解了数学运算和计算机实现之间的关系。

困难：然而，我也遇到了一些困难。首先，进位处理的细节容易出错，特别是在多次循环中需要确保每个元素的值保持在合理范围。其次，数组的边界管理也是一个挑战，尤其是在进行下标操作时，稍有不慎就可能导致数组越界或数据丢失。这些问题让我意识到在编写代码时需要更加细心，做好充分的测试和调试。

2.5 题目五

学生信息管理

2.2.1 题目描述

定义一个顺序表以存储学生信息（学号和姓名），实现对学生记录的基本操作，包括插入、删除和查找。

2.2.2 题目要求

首先，根据给定的学生总数输入相应的学生信息。然后，支持多种操作：在指定位置插入学生信息，删除指定位置的记录，分别根据姓名和学号查找学生，并输出相关信息。操作结束时，输出当前顺序表中学生的总人数。输入时需处理位置合法性，并在不合法时返回相应的错误码。

2.2.3 数据结构设计

```
定义结构体 student:
    字段:
        字符串 xuehao    // 学号
        字符串 name      // 姓名
```

Student 结构体用于存储学生信息，包含两个字段：xuehao 表示学生的学号，name 表示学生的姓名。这一结构便于在顺序表中管理和访问学生数据。

2.2.4 功能实现

定义类 studentlist:

字段:

学生指针 a_student // 动态数组存储学生信息
整数 most_num // 数组容量
整数 num // 当前学生数量

构造函数 studentlist(整数 most_num):

初始化 most_num 为参数值
初始化 num 为 0
分配内存给 a_student, 大小为 most_num

析构函数 ~studentlist():

释放 a_student 的内存

方法 input_student(整数 n):

如果 $n > \text{most_num}$, 则 $n = \text{most_num}$
对于 i 从 0 到 $n-1$:
 读取 a_student[i].xuehao 和 a_student[i].name
num += n

方法 insert(整数 index, 字符串 xuehao, 字符串 name):

如果 $\text{index} < 1$ 或 $\text{index} > \text{num} + 1$, 返回 -1 // 位置不合法

如果 $\text{num} \geq \text{most_num}$:

方法 insert(整数 index, 字符串 xuehao, 字符串 name):

如果 $\text{index} < 1$ 或 $\text{index} > \text{num} + 1$, 返回 -1 // 位置不合法

如果 $\text{num} \geq \text{most_num}$:

 扩展 most_num 100
 创建新数组 new_array 大小为 most_num
 复制 a_student 的内容到 new_array
 释放 a_student 的内存
 更新 a_student 为 new_array

对于 i 从 num 到 index:

$\text{a_student}[i] = \text{a_student}[i - 1]$ // 移动元素

设置 $\text{a_student}[\text{index} - 1].\text{xuehao} = \text{xuehao}$

设置 $\text{a_student}[\text{index} - 1].\text{name} = \text{name}$

num++

返回 0

方法 remove(整数 index):

如果 $\text{index} < 1$ 或 $\text{index} > \text{num}$, 返回 -1 // 位置不合法

对于 i 从 index 到 num - 1:

$\text{a_student}[i - 1] = \text{a_student}[i]$ // 移动元素

清空 $\text{a_student}[\text{num} - 1].\text{xuehao}$ 和 $\text{a_student}[\text{num} - 1].\text{name}$

num--

返回 0

```

方法 check_name(字符串 name):
    对于 i 从 0 到 num - 1:
        如果 name == a_student[i].name:
            输出 i + 1, a_student[i].xuehao, a_student[i].name
            返回
    输出 -1

方法 check_no(字符串 xuehao):
    对于 i 从 0 到 num - 1:
        如果 xuehao == a_student[i].xuehao:
            输出 i + 1, a_student[i].xuehao, a_student[i].name
            返回
    输出 -1

方法 end():
    返回 num

```

studentlist 类用于管理学生信息，包含动态数组 a_student 存储学生数据，most_num 表示数组容量，以及 num 表示当前学生数量。它提供构造函数初始化数组，析构函数释放内存，以及多种方法实现学生信息的输入、插入、删除、按姓名和学号查找，以及返回当前学生数量，确保操作合法性并动态扩展存储空间。

2.2.5 心得体会

心得：在实现 studentlist 类的过程中，我深刻体会到了动态数据结构的灵活性和复杂性。动态数组允许我们在需要时扩展存储空间，这对于处理不确定数量的学生信息非常重要。然而，这也带来了内存管理的挑战，特别是在进行扩展和释放内存时，确保没有内存泄漏或访问无效内存是一个需要谨慎对待的问题。

困难：在实现插入和删除操作时，保持数组的连续性和合法性也颇具挑战。例如，插入时需要移动元素，确保不会超出边界，而删除时则要正确地更新学生数量。这些操作要求我时刻关注索引的合法性，确保程序稳定运行。

3.实验总结

在实现级数和一元多项式的项目中，我深入理解了链表和数组这两种基本数据结构的特点与适用场景。使用链表来表示多项式的项可以有效地进行动态插入和删除操作，而数组则在访问和存储上提供了更高的效率。通过比较这两种结构，我学会了如何根据实际需求选择合适的数据结构，以提高程序的性能和可维护性。

在学生信息系统和轮转数组的实现过程中，我体会到了动态数组的优势与挑战。动态数组能够根据需要调整大小，非常适合处理不确定数量的数据，如学生信息。然而，内存的动态管理要求我在扩展和删除时确保数据的完整性和访问的安全性。此外，轮转数组的使用让我领悟到如何在特定场景下高效地处理循环数据，进一步加深了我对数据结构在实际问题中应用的理解。这些项目不仅提升了我的编程技能，也让我更好地掌握了数据结构的核心概念。